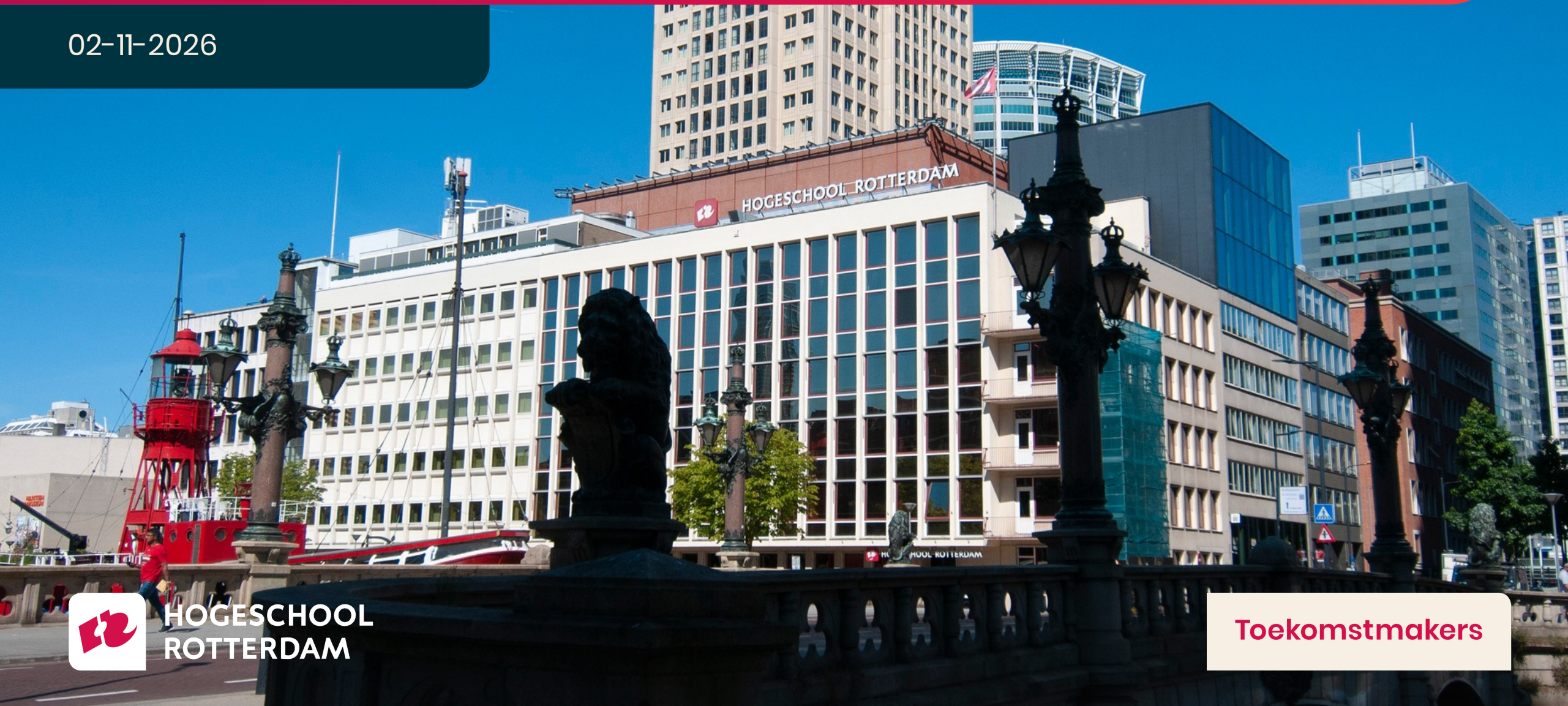
Technische Informatica: High-performance Computing

Les 22: Graphics engine: alles samenvoegen

02-11-2026

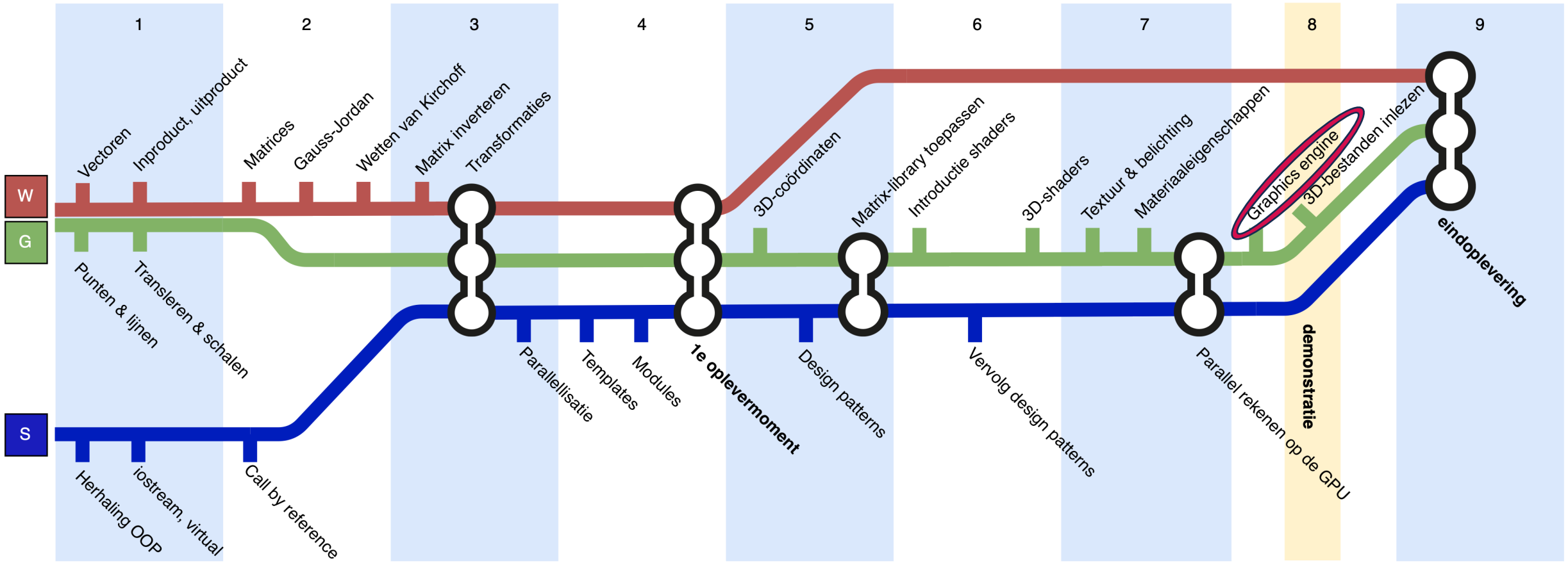


HOGESCHOOL ROTTERDAM

Toekomstmakers

Quiz

Metrokaart


1
2
3
4
5
6
7
8
9
W
G
S
Vectoren
Inproduct, uitproduct
Matrices
Gauss-Jordan
Wetten van Kirchoff
Matrix inverteren
Transformaties
3D-coördinaten
Matrix-library toepassen
Introductie shaders
3D-shaders
Textuur & belichting
Materiaaleigenschappen
Graphics engine
3D-bestanden inlezen
eindoplevering
Punten & lijnen
Transleren & schalen
Parallellisatie
Templates
Modules
1e oplevermoment
Design patterns
Vervolg design patterns
Parallel rekenen op de GPU
demonstratie
Herhaling OOP
iostream, virtual
Call by reference

Leerdoelen

De student kan:

1. bewerkingen met vectoren en matrices uitvoeren en deze toepassen in verschillende contexten zoals computer graphics en neurale netwerken.
2. object-georiënteerde programmeertechnieken toepassen in de programmeertaal C++.
3. shader-programma's schrijven en uitvoeren op een GPU in GLSL.
4. relevante design patterns onderbouwd selecteren en toepassen.



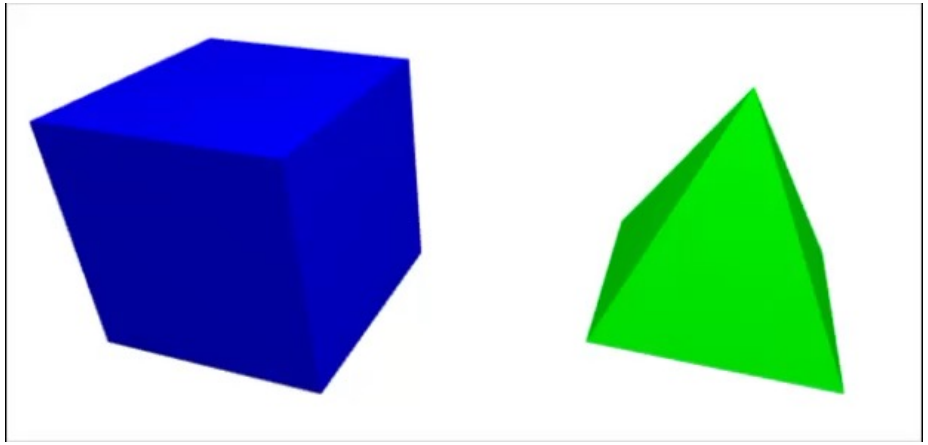
Planning

1. Meerdere vormen
2. Eigenschappen
3. Camera
4. Lichtbron
5. Muis-input



Meerdere vormen

- **Graphics engine**: meerdere vormen kunnen toevoegen
- Iedere vorm:
  - VBO: serie vertices (posities, normaalvectoren, evt. texture-coördinaten)
  - VAO: structuur van de VBO (welk datatype, welke getallen wat betekenen)
  - transformaties (rotaties, schaling, translatie)
  - eigenschappen (kleur, textuur, reflectie, enz.)
- Voor alle vormen: zelfde shader-programma
  - Data voor *uniforms* in de shader kan per vorm verschillen



C++ programma

Eenmalig:

1. Shaders compileren.
2. Voor iedere vorm:
    1. VAO aanmaken en binden
    2. VBO aanmaken en binden.
    3. de VBO vullen met data.
    4. *attributes* uit de vertex shader koppelen aan getallen in de VBO.

Telkens weer, voor iedere vorm:

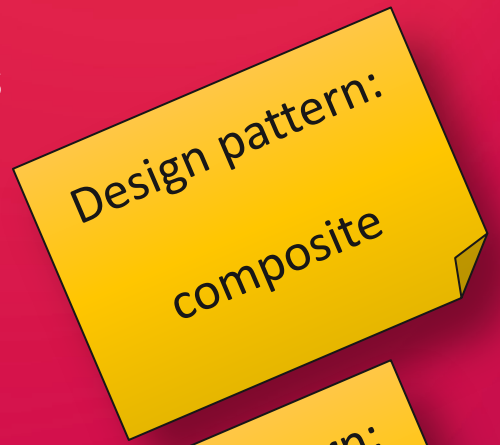
1. De VAO voor deze vorm binden.
2. De *uniforms* van data voorzien voor deze vorm.
3. De vorm tekenen met `glDrawArrays()`.

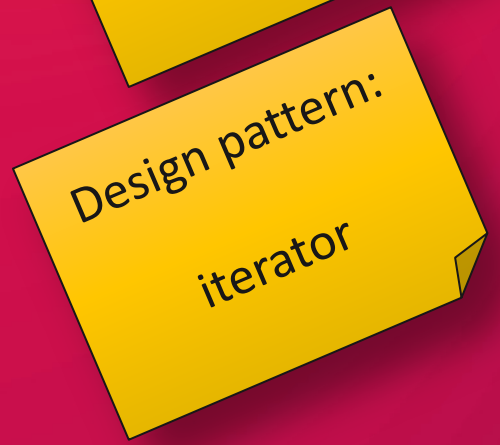


Opdracht 1: Meerdere vormen (object-georiënteerd)

- **Zie instructies op GitLab**.

- Voorbeeldcode laat zien hoe je twee vormen rendert (niet-OOP).
- Pas het programma aan zodat het OOP gebruikt.
- Maak een abstracte class `Shape` met concrete subclasses `Cube`, `Pyramid`, `Sphere` en `Cylinder`.
- De scene is *samengesteld* uit meerdere vormen die in een lijst staan. Het renderen van iedere vorm wordt uitbesteed aan de vorm zelf.
- De lijst zou bijv. ook een boomstructuur kunnen zijn. Als er maar een manier is om door alle vormen heen te lopen.

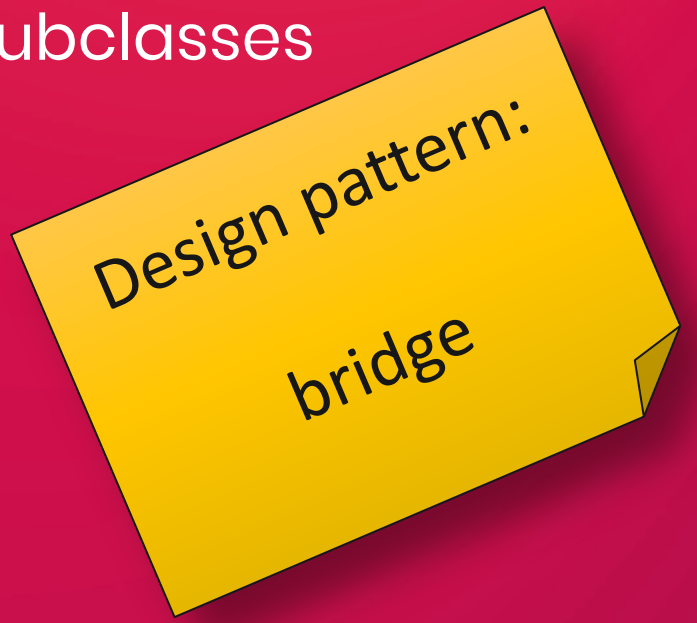




Opdracht 2: Eigenschappen

- **Zie instructies op GitLab**.

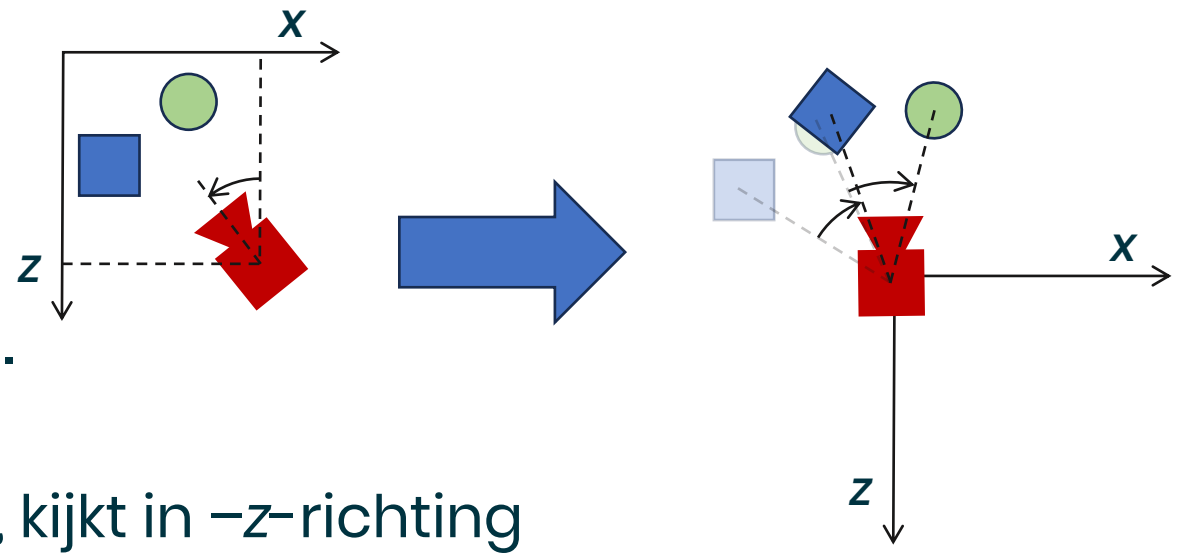
- Aparte classes voor blauwe kubus, blauwe bol, rode kubus, rode bol, enz.?
- Nee, één soort class voor vormen en één soort class voor eigenschappen.
- Maak een abstracte class `Property` met concrete subclasses als `Color`, `Texture`, `DiffuseReflection`, enz.
- Zorg dat je meerdere eigenschappen aan een vorm kan toevoegen.



Camera

- Tot nu toe: vaste projectie.
- Vanaf nu:
  - positie
  - rotatie
  - field of view

  van de camera kunnen instellen.



- Wiskundig gezien:
  - camera altijd in de oorsprong, kijkt in $-z$-richting
  - positie instellen door alle vormen te transleren in omgekeerde richting
  - rotatie instellen door alles rond de camera te laten roteren
- `Camera` is ook een soort transformatie ⇒ subclass van `Transform`

Camera-transformatie

- **Camera-transformatie** bestaat uit:
  1. Rotaties rond de *x*-, *y*-, en *z*-as.
  2. Translatie met een vector die tegengesteld is aan de positievector van de camera.
  3. Perspectiefprojectie met een gegeven *field of view*.
- Voor **posities**: alle 3 de onderdelen toepassen in vertex shader.
- Voor `fragPos` (voor lichtberekeningen): alleen 1 en 2.
- Voor **normaalvectoren**: alleen 1.

- Zorg dus dat de 3 onderdelen apart van elkaar opgeslagen worden in de `Camera`-class.



Opdracht 3: Camera-class

- **Zie instructies op GitLab**.

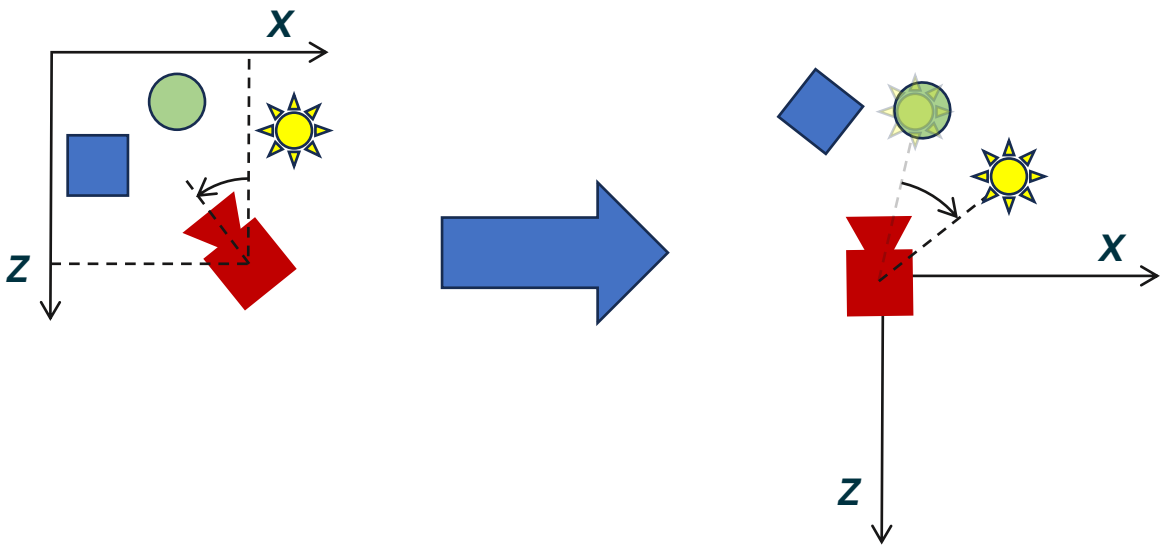
- Voeg een `Camera`-class (subclass van `Transform`) toe aan je programma uit de vorige opdracht.
- Voeg een constructor toe waar je de (x, y, z)-positie, rotatiehoeken rond de x-, y-, en z-as, en een *field of view* aan kan meegeven.
- De verschillende vormen hoeven nu niet meer apart “naar achteren” gezet te worden; dit wordt door de camera-transformatie gedaan.
- Pas de shaders en het C++-programma hierop aan.



Pauze (15min)

Lichtbron

- We willen voor de graphics engine de positie en de kleur van de lichtbron ook kunnen instellen.
- **Belangrijk**: de lichtbron moet ook getransformeerd worden door de camera-transformatie (zonder projectie).



Opdracht 4: Lichtbron-class

- **Zie instructies op GitLab**.

- Voeg een `Lightsource`-class toe aan je programma uit de vorige opdracht.
- Voeg een constructor toe waar je de (x, y, z)-positie en de kleur van de lichtbron kunt meegeven.
- Pas de shaders en het C++-programma hierop aan.

- Extra: kun je meerdere lichtbronnen toevoegen aan je programma?



Muis-input

- **Wens**: de camera (of andere onderdelen) met de muis besturen.
- In GLFW: `glfwGetCursorPos()` geeft *x*- en *y*-coördinaten van de muiscursor (voor rondkijken).
- Voor scroll-wiel (voor voor- en achteruit bewegen) is een callback-functie nodig:

```
double scroll;
void scrollCallBack(GLFWwindow* window, double xoffset, double yoffset) {
  scroll = yoffset;
}
```

- Instellen met:

```
glfwSetScrollCallback(window, scrollCallBack);
```



Opdracht 5: Muis-input

- **Zie instructies op GitLab**.

- Voeg muis-input toe aan je programma uit de vorige opdracht.
- Pas de camera-rotatie en -positie aan op basis van de muisbeweging en scroll-wiel.
- Zie https://www.glfw.org/docs/3.3/input_guide.html#input_mouse voor meer informatie.



Check-out

- Wat hebben we geleerd?
  - Hoe je meerdere vormen rendert met een shader.
  - Hoe je een camera en een lichtbron toevoegt aan je graphics engine.
  - Hoe je de muis kunt gebruiken om de camera te besturen.



Afsluiting

Verwerkingsopdracht:

- Zorg dat je alle 5 de opdrachten afhebt.

Voorbereidingsopdracht:

- Bedenk hoe je alle features van je graphics engine kan demonsteren.
- Installeer Blender op je computer.

Volgende keer:

- 3D-bestanden inlezen



HOGESCHOOL ROTTERDAM

Toekomstmakers

© 2025 Hogeschool Rotterdam, alle rechten voorbehouden